

QA Runbook — Dragoooo: Infrastructure, Security, Observability & Disaster Recovery

Assigned Engineer: Dragoooo

Assigned Roles & Boundaries: Single-Server VPS Production Fabric, Network Perimeter & Firewall Hardening, API Gateway Liveness & Abuse Defense, Observability Telemetry (Prometheus, Grafana, Loki), Container Chaos Resilience & Database Disaster Recovery

Architecture Grounding: [Blueprint §2.1 & §2.2](#), [Q026](#), [Q027](#), [Q047–Q053](#), [Q071–Q073](#), [Q081](#), [Q082](#), [Q096–Q098](#)

Primary Target Hosts & Endpoints: - Edge Domain: <https://stackandscale.org> - Public Status Portal:

<https://status.stackandscale.org> - API Health & Readiness Probes:

<https://api.stackandscale.org/health>, <https://api.stackandscale.org/ready> - Grafana Telemetry

Dashboard: <http://127.0.0.1:3001> (via encrypted SSH tunnel) - Prometheus Scrape Fabric:

<http://127.0.0.1:9090> - Production Host: [ubuntu@vps-5d4dfcb1](#) (OVHcloud VPS-2, Ubuntu 24.04 LTS)

1. Executive Context & System Architecture

Stack & Scale operates on an industrial **Cost-First Single-Server Production Topology** designed for maximum uptime and security without cloud provider vendor lock-in: - **OVHcloud VPS-2 Node:** 4 vCPU, 4 GB RAM, 50 GB NVMe storage running Ubuntu 24.04 LTS. - **16 Isolated Docker Containers:** Segregated across internal Docker bridge networks ([frontend](#), [backend](#), [database](#), [storage](#), [observability](#)). - **Edge Reverse Proxy (Caddy):** Handles TLS termination using Cloudflare Origin CA certificates; all public web traffic is routed through Cloudflare's WAF and DDoS mitigation layers. - **Unified Observability Fabric:** Prometheus (metrics engine), Grafana (visualization), Loki (log aggregator), Promtail (log shipper), cAdvisor (container telemetry), and Node Exporter (host metrics). - **Hardened Kernel Sandbox:** Rootless containers where applicable, strict [cap_drop](#) capabilities, and non-routable database/storage ports.

As **Dragoooo**, you are the platform reliability, security, and infrastructure authority. Your mission is to verify that internal ports are completely hidden from the internet, test API rate-limiting defenses, audit live telemetry in Grafana and Loki, execute a live container crash recovery chaos drill, verify database backups, and assemble the Master Production Launch Sign-Off.

2. Testing Environment & Prerequisites

- **Local Terminal:** With [ssh](#), [curl](#), [nc](#) (netcat), or [nmap](#).
 - **SSH Access:** [ssh ubuntu@vps-5d4dfcb1](#) using your authorized private key.
 - **Browser:** To access Grafana dashboards via encrypted port-forwarding.
 - **Handoff from Hanzala Khan:** Ensure you have received confirmation of the portal e-signatures and ClamAV malware test events.
-

3. Step-by-Step Execution Suites

Test Suite 4.1: Edge SSL/TLS & Perimeter Network Hardening

Context:

Per **Q026**, **Q081**, and **Q096**, public traffic is allowed ONLY via ports 80, 443 (Cloudflare proxy) and 22 (SSH). Database, storage, and scanner ports must NEVER be reachable from the outside.

Step-by-Step Instructions:

Part A: Cloudflare TLS 1.3 Full (strict) & HSTS Verification

1. From your local terminal, inspect the TLS handshake on the primary domain:

```
curl -Iv https://stackandscale.org 2>&1 | grep -i -E "HTTP/|SSL
certificate|server:|strict-transport"
```

2. Verify Cryptographic Headers:

- **Protocol:** `HTTP/2` or `HTTP/3`.
- **TLS Version:** `TLSv1.3`.
- **Server:** `cloudflare`.
- **HSTS Header:** `strict-transport-security: max-age=31536000; includeSubDomains; preload` is present.

3. Test HTTP to HTTPS redirection:

```
curl -I http://stackandscale.org
```

- **Verification:** Returns HTTP `301 Moved Permanently` with `Location: https://stackandscale.org/`.

Part B: Public Port Scan & Internal Denial Test (Critical Security Gate)

1. Find your VPS public IP address (run `dig +short stackandscale.org` or inspect host network info).
2. From your local machine, run netcat connection probes against the internal service ports:

```
TARGET_IP="<YOUR_VPS_PUBLIC_IP>"

echo "--- Testing PostgreSQL Port 5432 ---"
nc -zv -w 3 "$TARGET_IP" 5432 2>&1 || echo "Port 5432 BLOCKED (Passed)"

echo "--- Testing MinIO S3 Port 9000 ---"
nc -zv -w 3 "$TARGET_IP" 9000 2>&1 || echo "Port 9000 BLOCKED (Passed)"

echo "--- Testing MinIO Console Port 9001 ---"
nc -zv -w 3 "$TARGET_IP" 9001 2>&1 || echo "Port 9001 BLOCKED (Passed)"

echo "--- Testing ClamAV Scanner Port 3310 ---"
nc -zv -w 3 "$TARGET_IP" 3310 2>&1 || echo "Port 3310 BLOCKED (Passed)"

echo "--- Testing API Internal Port 3000 ---"
nc -zv -w 3 "$TARGET_IP" 3000 2>&1 || echo "Port 3000 BLOCKED (Passed)"
```

3. Verify Strict Security Results:

- Every single test above MUST output `Connection timed out`, `Connection refused`, or `BLOCKED (Passed)`.

- **Security Failure Condition:** If any internal port (5432, 9000, 9001, 3310, 3000) reports `open`, immediately flag a perimeter breach. Only ports 80, 443, and 22 are permitted.

Test Suite 4.2: API Probes & Rate Limiting Abuse Defense

Context:

Per **Q073** and **Q096**, the API gateway provides standard health probes and dynamically rate-limits incoming traffic to prevent abuse.

Step-by-Step Instructions:

Part A: API Probes

1. Run a request against the liveness probe:

```
curl -i https://api.stackandscale.org/health
```

- **Verification:** Status `200 OK`, Content-Type `application/json`, payload:

```
{"status":"ok"}
```

2. Run a request against the readiness probe:

```
curl -i https://api.stackandscale.org/ready
```

- **Verification:** Status `200 OK`, confirming database connection pool, outbox worker, and disk paths are fully initialized.

Part B: Rate Limiting Abuse Defense Test

1. Execute a rapid burst of 120 HTTP GET requests against the API gateway within 5 seconds:

```
for i in {1..120}; do  
  curl -s -o /dev/null -w "%{http_code}\n" https://api.stackandscale.org/health  
done | sort | uniq -c
```

2. Analyze the Distribution:

- The output will resemble:

```
60 200  
60 429
```

- **Verification:** Initial requests return `200 OK`. Once the quota is reached, excess requests are throttled with HTTP `429 Too Many Requests`.

3. Inspect the response headers:

```
curl -i https://api.stackandscale.org/health
```

- Verify presence of header: `retry-after: [seconds]`.

Test Suite 4.3: Observability Stack (Prometheus, Grafana, Loki)

Context:

Per **Q049** and **Q098**, the unified observability fabric provides real-time visibility into CPU, memory, container health, and structured application logs.

Step-by-Step Instructions:

Part A: Establish Secure Grafana SSH Tunnel

1. In your local terminal, open a port-forwarded SSH session to the VPS:

```
ssh -L 3001:127.0.0.1:3001 ubuntu@vps-5d4dfcb1
```

2. In your local browser, navigate to: <http://localhost:3001>.
3. Sign in to Grafana:

- **Username:** `admin`
- **Password:** Read the password on the VPS:

```
sudo cat /opt/stack-and-scale/secrets/grafana-admin-password
```

Part B: Verify System Resource Telemetry Dashboard

1. Inside Grafana, navigate to **Dashboards** → **Stack & Scale Platform Overview**.
2. Audit the live performance gauges across all 16 containers:
 - **Host CPU Utilization:** Must be below **15%** (typically idling between 4% – 8%).
 - **Host RAM Utilization:** Must be below **40%** (~1.2 GB – 1.6 GB used of 4.0 GB total).
 - **Disk NVMe Usage:** Must be below **35%** (healthy headroom on 50GB drive).
 - **Network I/O:** Smooth baseline with zero abnormal egress spikes.

Part C: Prometheus Scrape Targets Audit

1. Open a new terminal tab and inspect Prometheus targets:

```
ssh ubuntu@vps-5d4dfcb1 'curl -s http://127.0.0.1:9090/api/v1/targets | grep -o "\"health\": \"[^\"]*\"\" | sort | uniq -c'
```

2. Verification:

- All targets report `"health": "up"`.
- Scrape jobs: `api`, `cAdvisor` (container metrics), and `node-exporter` (host metrics) are all green.

Part D: Structured Log Verification in Loki

1. In Grafana, click the **Explore** compass icon in the left navigation menu.
2. Ensure the data source is set to **Loki**.
3. **Query 1: Inspect ClamAV Scanning Logs from Hanzala Khan's Test:**
 - In the query input, enter:

```
{container_name="stack-and-scale-production-clamav-1"}
```

- Click **Run query**:

- **Verification:** Locate the log entry generated during Hanzala Khan's EICAR test: `... FOUND: Eicar-Test-Signature` Confirming ClamAV caught and logged the malware event.

4. Query 2: Inspect API Request Stream:

- Enter query:

```
{container_name="stack-and-scale-production-api-1"} |= "POST /api/leads"
```

- Click **Run query**:

- **Verification:** Locate the log entry corresponding to Muhammad Saad Khan's lead submission for `Alex Mercer`, including response status `201` and execution latency (`< 40ms`).

Test Suite 4.4: Chaos Drill — Self-Healing Container Auto-Recovery

Context:

Per **Q027**, **Q048**, and **Q098**, the platform uses Docker's restart policies (`restart: unless-stopped`) to automatically recover from fatal process crashes with zero operator intervention.

Step-by-Step Instructions:

1. In your SSH session on the VPS, check the uptime of the live API container:

```
docker ps --filter "name=api" --format "table {{.Names}}\t{{.Status}}"
```

2. **Simulate a Fatal Container Crash:**

- Execute an ungraceful kill signal:

```
docker kill stack-and-scale-production-api-1
```

3. **Observe Automated Docker Resurrection:**

- Immediately watch the container state:

```
for i in {1..8}; do
  docker ps --filter "name=api" --format "{{.Names}}: {{.Status}}"
  sleep 1
done
```

4. **Analyze the Recovery:**

- Within 2–4 seconds, Docker detects process termination.
- Status transitions to `Up 3 seconds (health: starting)` → `Up 8 seconds (healthy)`.

5. From your local machine, test the API health probe:

```
curl -i https://api.stackandscale.org/health
```

- **Verification:** Returns `200 OK`. The service has self-healed.

Test Suite 4.5: Database Backup Execution & Integrity Verification

Context:

Per **Q050** and **Q097**, database snapshots are executed via automated scripts and dumped to encrypted compressed SQL archives.

Step-by-Step Instructions:

1. In your SSH session on the VPS, navigate to the deployment root:

```
cd /opt/stack-and-scale
```

2. Execute the Platform Backup Script:

```
bash scripts/backup.sh
```

3. Inspect the Backup Artifacts:

- List the contents of the backup directory:

```
ls -lh /opt/stack-and-scale/backup/
```

- Verify a new file exists: `stack_and_scale_backup_YYYYMMDD_HHMMSS.sql.gz` (or `.sql`).
- Check the file size: **Must be > 500 KB** (confirming real database schema and data are present, not an empty stub).

4. Verify SQL Dump Integrity:

- Inspect the file header:

```
zcat /opt/stack-and-scale/backup/stack_and_scale_backup_*.sql.gz 2>/dev/null |  
head -n 25 || head -n 25 /opt/stack-and-scale/backup/*.sql
```

- **Verification:** Header confirms valid PostgreSQL export:

```
-- PostgreSQL database dump  
-- Dumped from database version 16.x  
-- Dumped by pg_dump version 16.x
```

5. Confirm that OVHcloud automated daily VPS snapshots are active in the OVH Cloud Manager dashboard as an independent host-level disaster recovery safeguard.

4. Master Team Sign-Off Assembly & Launch Gate

Once all 4 engineers have completed their respective runbooks and signed their sheets, **Dragooooo** collects the signed checklists and executes the master sign-off:

```
=====
STACK & SCALE PRODUCTION LAUNCH SIGN-OFF SUMMARY
=====
Environment: OVHcloud Production Host (vps-5d4dfcb1)
Release Digest / Commit: [Current Git Commit SHA]
Sign-Off Timestamp: [Record UTC Date & Time]

Member 1 (Muhammad Saad Khan) – Marketing & CMS:      [ ] APPROVED (Signature:
_____)
Member 2 (Talha Shams)      – Identity & Staff CRM:    [ ] APPROVED (Signature:
_____)
Member 3 (Hanzala Khan)     – Client Portal & Storage: [ ] APPROVED (Signature:
_____)
Member 4 (Dragooooo)        – Infra & Observability:   [ ] APPROVED (Signature:
_____)

FINAL VERDICT: [ ] 100% PRODUCTION READY FOR PUBLIC LAUNCH
=====
```

5. Official Dragoooo Sign-Off Sheet

Test Case	Description	Pass / Fail	Operator Signature	Timestamp
TC-4.1	Cloudflare TLS 1.3 Full (strict) & HSTS response headers confirmed	☐ Pass / ☐ Fail	_____	_____
TC-4.2	Internal ports 5432, 9000, 9001, 3310, 3000 completely blocked	☐ Pass / ☐ Fail	_____	_____
TC-4.3	API liveness (<code>/health</code>) and readiness (<code>/ready</code>) return HTTP 200	☐ Pass / ☐ Fail	_____	_____
TC-4.4	Rate limiter throttles burst traffic with HTTP 429 & <code>retry-after</code>	☐ Pass / ☐ Fail	_____	_____
TC-4.5	Grafana host telemetry confirms CPU < 15% and RAM < 40%	☐ Pass / ☐ Fail	_____	_____
TC-4.6	Prometheus scrape targets (<code>api</code> , <code>cAdvisor</code> , <code>node-exporter</code>) all UP	☐ Pass / ☐ Fail	_____	_____
TC-4.7	Loki indexes structured application logs and ClamAV malware alerts	☐ Pass / ☐ Fail	_____	_____
TC-4.8	Chaos Drill: killed API container self-heals healthy in < 10 seconds	☐ Pass / ☐ Fail	_____	_____
TC-4.9	Database backup script produces valid, uncorrupted SQL dump (> 500KB)	☐ Pass / ☐ Fail	_____	_____